🔷 What is Pimcore?

Pimcore is an **open-source digital experience platform (DXP)** that helps businesses **manage, enrich, and deliver digital data** across multiple channels. It's built on the **Symfony PHP framework** and is highly flexible and scalable.

It combines multiple systems into one platform:

🔷 Main Modules of Pimcore

1.**PIM (Product Information Management)**

- Central hub for all product-related data.
- Helps manage attributes (size, color, material, SKU), variants, categories, and relationships.
- Ensures **consistent product data** across websites, apps, and marketplaces.

2.**DAM (Digital Asset Management)**

- Central repository for all media assets (images, videos, PDFs, etc.).
- Provides metadata management, version control, tagging, and rights management.
- Assets can be directly linked to products, categories, or content pages.

3.**CMS (Content Management System)**

- Manages website content (pages, blogs, landing pages).
- WYSIWYG editing and drag-and-drop layouts.
- Supports both **traditional CMS** and **headless CMS** approaches.

4.**MDM (Master Data Management)**

- Extends PIM to handle all types of data (customers, suppliers, vendors, locations, etc.).

- Ensures a **single source of truth** for critical business data.

5.**eCommerce Framework**

- Flexible framework to create product catalogs, carts, and checkout processes.
- Can integrate with ERP, CRM, and payment systems.
- Supports personalized shopping experiences.

🔷 Setting Up a Pimcore Development Environment

Here's a simple way to get Pimcore running:

1. System Requirements

- PHP ≥ 8.1
- MySQL ≥ 8.0 or MariaDB ≥ 10.4
- Composer (PHP dependency manager)
- Node.js & npm/yarn (for front-end tools, optional)
- Web server (Apache or Nginx)

---

2. Installation Steps

Option A: Using Composer (recommended)

```
# Create a new project

composercreate-projectpimcore/skeletonmy-project


# Move into the project folder

cdmy-project


# Install dependencies
```

```
composerinstall
```

Option B: Using Docker (easier for beginners)

```
gitclonehttps://github.com/pimcore/skeletonmy-project

cdmy-project


# Start with docker-compose

docker-composeup-d
```

---

3. Database Setup

Run the Pimcore installation command:

```
bin/consolepimcore:install
```

This will create the database schema and basic configuration.

---

4. Access Pimcore

- Open browser → `http://localhost`
- Login at `/admin`
- Default credentials (you'll set them during install)

✅ Now you have Pimcore running with access to all its modules.

🔷 Data Objects, Assets, and Documents in Pimcore

Think of Pimcore as having **three core building blocks**:

1. **Data Objects**
    - Structured data models (like **Products, Categories, Customers, Stores**).
    - Each Object Class has fields (text, number, image, relation, etc.).
    - Example: A *Product* object might have: Name, SKU, Price, Image.
2. **Assets**
    - Files like **images, videos, PDFs, documents**.
    - Stored in the Digital Asset Management (DAM).
    - Can be linked to Data Objects (e.g., product image).
3. **Documents**
    - Used for **website content/pages** (like CMS).
    - Example: A landing page, a blog post, or a content block.

👉 Rule of thumb:

- **Objects = structured data**
- **Assets = media files**
- **Documents = content pages**

🔷 Creating a New Data Object Class (Example: Product)

Step 1: Open Class Definitions

- Log in to **Pimcore Admin** (`/admin`).
- Go to: **Settings → Data Objects → Classes**.
- Click **+ Add Class**.

Step 2: Define the Class

- Name it: `Product`
- Add a description if you want.
- Save.

Step 3: Add Fields (Attributes)

Inside the class editor, you can add fields:

- **Name** → `Input` (single-line text)
- **SKU** → `Input` (unique identifier)
- **Price** → `Numeric`
- **Description** → `WYSIWYG` editor
- **Image** → `Image` (relation to an Asset)
- **Category** → `Many-to-One Relation` (to another class like Category)

⚡ You can organize fields into **panels, fieldsets, and layouts** for better UI.

Step 4: Save & Publish the Class

- Click **Save & Publish**.
- Pimcore will now generate this Data Object Class.

Step 5: Create a Product Instance

- Go to **Data Objects** in the sidebar.
- Right-click in the tree → **Add Object** → **Product**.
- Fill in the fields (Name, SKU, Price, etc.).
- Upload an image from Assets and link it.
- Save.

✅ Now you have your first **Product Data Object**.

🔷 Example Use Case

- A **Product** object has structured data (SKU, Price, Description).
- Its **Image** is stored in **Assets**.
- It appears on a **Product Detail Page (Document)** in your website CMS.

👉 Next Steps: You could build a **Product Catalog** using these objects, and display them in a Document with editable templates.

🔷 Step 1: Product Class Setup (Attributes)

In **Settings** → **Data Objects** → **Classes** → **Product**, you can add attributes like:

- **Basic Info**
  - Name → `Input`
  - SKU → `Input (unique)`
  - Description → `WYSIWYG`
- **Commercial Data**
  - Price → `Numeric`
  - Stock → `Numeric`
  - Active → `Checkbox`
- **Media**
  - Image → `Image` (linked from Assets)
  - Gallery → `Image Gallery`

👉 Attributes = **fields that describe a product**.

🔷 Step 2: Product Categories

Create a Category Class:

1. Go to **Settings** → **Data Objects** → **Classes** → **Add Class** → **Category**.
2. Add fields like:
    - Category Name → `Input`
    - Description → `Textarea`
    - Parent Category → (optional relation to Category itself for hierarchy).

Use Relation in Product:

- In **Product class**, add a field:
    - **Category** → `Many-to-One Relation` (linked to `Category` class).

👉 Now each Product can belong to one Category. 👉 For multiple categories, you could use **Many-to-Many Relation**.

🔷 Step 3: Relationships Between Products

Let's say we want to manage **related products** (e.g., accessories, upsells, bundles).

Add Relation Fields in Product Class:

- **Related Products** → `Many-to-Many Object Relation` (linked to `Product`).
- **Alternative Products** → `Many-to-Many Object Relation` (linked to `Product`).

This allows you to connect one product with others.

Example:

- Product A (Laptop) → Related Products: Mouse, Bag.
- Product B (Phone) → Alternative Products: Phone 2, Phone 3.

🔷 Step 4: Managing Data in the Admin Panel

1. Go to **Data Objects** → right-click → **Add Object** → **Category**.
    - Example: Category = “Electronics”.
    - Another Category = “Accessories”.
2. Go to **Data Objects** → Add **Product**.
    - Fill attributes: Name = “Laptop X1”, SKU = “LAP123”, Price = 1200.
    - Assign **Category** = “Electronics”.
    - Upload an image from Assets.
3. Add another **Product**: “Wireless Mouse”.
    - Category = “Accessories”.
    - Price = 30.
4. Open **Laptop X1** → in **Related Products**, select “Wireless Mouse”.

✅ Now you have products connected by categories and relationships.

🔷 Step 5: Example Use Case

- In a **Product Detail Page**:
    - Show Product Attributes (Name, Price, Description).
    - Show Product Image (from Assets).
    - Show Category (e.g., Electronics).
    - Show Related Products (Wireless Mouse, Bag, etc.).

This is how Pimcore lets you build **rich product catalogs** with structured data.

🔷 Step 1: Upload Assets

1. In **Admin Panel**, go to the left sidebar → **Assets**.
2. Right-click → **Add Folder** (e.g., `Product Images`).
3. Inside the folder, drag & drop files (images, videos, PDFs) from your computer.
    - Example: Upload `laptop.jpg`, `manual.pdf`, `promo.mp4`.

✅ Assets are now stored in Pimcore's DAM.

🔹 Step 2: Tag & Metadata

Each asset can have **metadata and tags**:

1. Select an uploaded asset (`laptop.jpg`).
2. Go to **Properties / Metadata tab**.
    - Add fields like:
        - `Alt Text` = "Laptop X1 Product Image"
        - `Copyright` = "© Company 2025"
        - `Tags` = Laptop, Electronics, Product

👉 This makes assets **searchable and reusable** across projects.

🔹 Step 3: Organize Assets

- Use **folders** (e.g., `Products → Electronics → Laptops`).
- Use **tags** for quick search (e.g., all "PDF manuals").
- Use **versions** (Pimcore keeps older versions when you re-upload).

🔹 Step 4: Connect Assets with Products

There are 2 main ways:

A. Image / Asset Field in Product Class

1. Go to **Settings** → **Data Objects** → **Classes** → **Product**.
2. Add a new field:
    - **Image** → `Image` (for one image).
    - **Gallery** → `Image Gallery` (for multiple images).
    - **File** → `Many-to-Many Object Relation with Asset` (for PDFs, videos, etc.).
3. Save & Publish class.

Now, when you open a **Product object** (e.g., Laptop X1), you'll see these fields. You can **drag & drop assets** (images, PDFs, videos) directly into them.

B. Linking from the Asset Side

- Open an Asset (`laptop.jpg`).
- In the **Relations tab**, you can see which Products are using this asset.
- This way, you can track usage across the system.

🔷 Step 5: Example

Let's say we have:

- **Product:** Laptop X1
    - SKU: LAP123
    - Price: 1200
    - Image: `laptop.jpg` (Asset)
    - Manual: `manual.pdf` (Asset)

Now:

- In the frontend (CMS Document or API), you can fetch and display:
    - Laptop Name + Price (from Product Object).

- Product Image (from Asset).
- Downloadable Manual (from Asset).

✅ With this setup, you get:

- Centralized **media library**.
- Clean **organization** (folders, tags, metadata).
- Strong **connections** between products and assets.

🔹 Step 1: Understanding Documents in Pimcore

There are **three main types** of Documents:

1. **Page** → A web page (e.g., Homepage, About Us, Product Detail).
2. **Snippet** → A reusable piece of content (e.g., Footer, Promo Banner).
3. **Email** → Templates for transactional/marketing emails.

👉 For this example, we'll build a **content page** using a **Page Document**.

🔹 Step 2: Create a New Page

1. In **Admin Panel** → **Documents** (left sidebar).
2. Right-click on the root (or a folder) → **Add Document** → **Page**.
3. Enter details:
   - Name = `about-us`
   - Title = "About Us"
   - Template = `default.html.twig` (you can use an existing one).
4. Save & Publish.

✅ You now have a new page accessible at `/about-us`.

🔹 Step 3: Use Editable Areas (WYSIWYG & Others)

In the Twig template (`default.html.twig`), you define **editable areas** like this:

```
{% extends "layout.html.twig" %}

{% block content %}
    <h1>{% pimcore_input('headline', {'width': 600}) %}</h1>
    <div>
        {% pimcore_wysiwyg('content', {'height': 300}) %}
    </div>
    <div>
        {% pimcore_image('mainImage', {'thumbnail': 'content'}) %}
    </div>
{% endblock %}
```

- `pimcore_input` → single line editable text (e.g., headline).
- `pimcore_wysiwyg` → rich text editor (content block).
- `pimcore_image` → an image that editors can replace from DAM.

🔹 Step 4: Edit Content in Admin

1. Go to **Documents** → **About Us** → **Edit**.
2. You'll see the WYSIWYG editor and editable fields (Headline, Content, Image).
3. Enter text in the WYSIWYG (e.g., company story, mission statement).
4. Drag & drop an image from Assets.
5. Save & Publish.

✅ Non-technical users can now update text & images without touching code.

🔹 Step 5: Organize Content Pages

- Use **folders** under Documents (e.g., `/products`, `/company`).
- Use **Snippets** for reusable blocks (like a newsletter signup box).
- Use **Layouts** (base Twig templates) for consistency across pages.

🔹 Example Use Case

- `/about-us` → has headline, text, and image defined by editors.
- `/products/laptop-x1` → can use the same template, but load product data dynamically from Data Objects.

🔹 1. How Pimcore Acts as a Headless CMS

- **Traditional CMS** → Renders content inside Pimcore templates (Twig, PHP).
- **Headless CMS** → Pimcore only stores & manages content; you fetch it via APIs and render it in **React, Angular, Vue, mobile apps, or even IoT devices**.

👉 This is enabled by **Pimcore Data Hub**, which provides **REST** and **GraphQL APIs** for structured access.

🔹 2. Setting up Data Hub

1. In Pimcore Admin → **Tools** → **Data Hub** → **Configurations**.
2. Add a new **GraphQL or REST configuration**.
3. Choose which Data Objects / Classes to expose (e.g., `Product`, `Category`).
4. Define which fields are accessible (to avoid exposing everything).
5. Save & Publish the Data Hub config.

✅ Now you have an endpoint:

- GraphQL → `/graphql/<configName>`
- REST → `/webservice/rest/<configName>`

🔹 3. Example: Fetching Products via GraphQL

Suppose we have a **Product** class with fields: `name`, `sku`, `price`, `description`.

Query:

```
query {
  getProduct(id: 123) {
    name
    sku
    price
    description
  }
}
```

Response:

```
{
  "data": {
    "getProduct": {
      "name": "Laptop X1",
      "sku": "LAP123",
      "price": 1200,
      "description": "A lightweight business laptop"
    }
  }
}
```

🔷 4. Example: Fetching Content Page via GraphQL

If you expose **Documents (Pages)**:

Query:

```
query {
  getDocument(id: 45) {
    fullPath
    title
    properties {
      key
      value
    }
```

```
  }
}
```

Response:

```
{
  "data": {
    "getDocument": {
      "fullPath": "/about-us",
      "title": "About Us",
      "properties": [
        { "key": "seoTitle", "value": "About Our Company" },
        { "key": "seoDescription", "value": "Learn about our mission" }
      ]
    }
  }
}
```

🔷 5. Example: REST API (Products)

If you use **REST** instead of GraphQL:

Request:

```
GET /webservice/rest/Product/123
```

Response:

```
{
  "id": 123,
  "type": "object",
  "className": "Product",
  "data": {
    "name": "Laptop X1",
    "sku": "LAP123",
    "price": 1200,
    "description": "A lightweight business laptop"
```

```
    }
}
```

🔷 6. Real-World Use Cases

- **Headless Website** → Pimcore manages data, React/Vue frontend fetches via GraphQL.
- **Mobile App** → iOS/Android fetch product catalog via REST.
- **Marketplaces** → Syndicate Pimcore product data to Amazon, Shopify, etc.
- **Omnichannel** → Same content powers website, app, in-store displays.

✅ This is why Pimcore is powerful — you get **both traditional CMS & headless CMS** in one.

🔷 1. What Are Workflows in Pimcore?

- **Workflows** = define a series of **states** (e.g., Draft → Review → Approved → Published).
- They help manage **who can do what** at each stage.
- They can be applied to **Data Objects, Assets, or Documents**.
- Built on Symfony's **workflow component**.

👉 Example use cases:

- Product content approval before publishing.
- Marketing images review before being used.
- Multi-language content validation.

🔷 2. Workflow Components

A workflow consists of:

1. **States** → stages of the process (Draft, Review, Approved).
2. **Transitions** → rules to move from one state to another.

3. **Roles & Permissions** → who can execute transitions.
4. **UI Actions** → appear as buttons in the Pimcore admin panel.

🔷 3. Example: Product Approval Workflow

Goal:

A product should move from **Draft** → **Review** → **Approved** → **Published**.

Step 1: Define Workflow in YAML

Create a config file: `var/config/workflow.yaml`

```yaml
workflows:
  product_approval:
    label: "Product Approval Workflow"
    type: 'state_machine'
    supports:
      - 'Pimcore\Model\DataObject\Product'
    places:
      - draft
      - review
      - approved
      - published
    initial_place: draft
    transitions:
      submit_for_review:
        from: draft
        to: review
      approve:
        from: review
        to: approved
      publish:
        from: approved
        to: published
      send_back:
        from: review
        to: draft
    metadata:
      submit_for_review:
        label: "Submit for Review"
      approve:
        label: "Approve"
```

```
    publish:
      label: "Publish"
    send_back:
      label: "Send Back to Draft"
```

Step 2: Assign Workflow to Users

- In **Settings** → **Users/Roles**, assign workflow permissions.
- Example:
  - **Content Editors** can move Draft → Review.
  - **Managers** can move Review → Approved.
  - **Admins** can Publish.

Step 3: Workflow in Action

1. A new **Product** is created → starts in **Draft**.
2. Editor clicks **Submit for Review**.
3. Manager reviews the content → clicks **Approve**.
4. Admin clicks **Publish** → Product goes live.

In the **Product editor screen**, users will see workflow buttons depending on their permissions.

🔷 4. Benefits

- Ensures **quality control**.
- Provides **audit trails** (who approved what, when).
- Makes content governance easier for teams.

✅ With this workflow, product data goes through a proper pipeline before being published.

🔷 1. How Pimcore Handles Localization

- **System Languages**: You configure which languages your project supports (e.g., `en`, `de`, `fr`).
- **Data Objects** (like Products): Fields can be made **localized** (translatable per language).
- **Documents** (CMS pages): You can create **language-specific page trees**.
- **Assets**: Can have **metadata translations** (alt text, titles, captions).

👉 This allows a single Pimcore instance to manage content for **multiple regions & languages**.

🔷 2. Configure System Languages

1. In Admin → **Settings** → **System** → **Localization & Internationalization**.
2. Add languages (e.g., `en`, `de`, `fr`).
3. Save.

✅ Pimcore now knows which languages you support.

🔷 3. Multilingual Product Data (Data Objects)

Step 1: Update Product Class

- Go to **Settings** → **Data Objects** → **Classes** → **Product**.
- Add **Localized Fields** container.
- Inside it, add fields like:
  - Name (Input)
  - Description (WYSIWYG)

Step 2: Fill Data Per Language

- Open a Product (e.g., Laptop X1).
- You'll now see language tabs (`EN`, `DE`, `FR`) at the top.
- Enter translations per language:
  - **EN** → Name = "Laptop X1", Description = "Lightweight business laptop".
  - **DE** → Name = "Laptop X1", Description = "Leichter Business-Laptop".
  - **FR** → Name = "Laptop X1", Description = "Ordinateur portable léger pour entreprise".

✅ The same product object now holds all translations.

🔹 4. Multilingual Content Pages (Documents)

Step 1: Create a Language Root

- In **Documents**, right-click root → **Add Folder** → name = `en`.
- Do the same for `de`, `fr`.

Step 2: Create Translated Pages

- Inside `/en`, add **About Us** page.
- Inside `/de`, add **Über uns** page.
- Inside `/fr`, add **À propos de nous** page.

👉 Each language folder = one localized website tree.

🔹 5. Switching Languages on the Frontend

- Pimcore auto-detects language via **URL prefix** (`/en/about-us`, `/de/ueber-uns`).
- You can add a **language switcher** in Twig:

```
<ul>
  <li><a href="{{ pimcore_url(document, {'_locale': 'en'}) }}">English</a>
</li>
  <li><a href="{{ pimcore_url(document, {'_locale': 'de'}) }}">Deutsch</a>
</li>
  <li><a href="{{ pimcore_url(document, {'_locale': 'fr'}) }}">Français</a>
</li>
</ul>
```

🔹 6. Assets Metadata Translations

- Select an image in **Assets**.
- Go to **Metadata tab** → enter translated values per language (e.g., Alt Text in English, German, French).

🔹 7. Example Use Case

- `/en/products/laptop-x1` → shows product in English.
- `/de/produkte/laptop-x1` → same product, German translation.
- `/fr/produits/laptop-x1` → same product, French translation.

All powered by **one Pimcore instance** with structured multilingual data.

✅ This way, Pimcore helps you manage **global catalogs & websites** without duplication.

🔹 1. What is Pimcore's eCommerce Framework?

- It's not a ready-made "shop out of the box" like Shopify.
- Instead, it's a **flexible toolkit** to build custom e-commerce solutions.
- You model products in **Data Objects**, then use the framework to:

- Organize them into a **catalog**.
- Expose them in APIs or frontend templates.
- Add pricing, availability, cart & checkout logic.

🔷 2. Setup Requirements

Make sure you have:

- A **Product Class** (already created earlier: Name, SKU, Price, Category, Image).
- **Categories Class** (to organize products).
- At least 2–3 products added in **Data Objects**.

🔷 3. Install E-Commerce Bundle (if not included)

```
composer require pimcore/pimcore-ecommerce-framework
```

Enable it in `config/bundles.php` if not auto-registered.

🔷 4. Configure Products for the Catalog

Step 1: Create Category Structure

1. Go to **Data Objects → Categories**.
2. Add:
   - Electronics
   - Accessories
   - Home Appliances

Step 2: Assign Products

- Product: Laptop X1 → Category = Electronics.
- Product: Wireless Mouse → Category = Accessories.

🔷 5. Define Product Index (Searchable Catalog)

Pimcore uses an **index service** (similar to Elasticsearch but works with MySQL too). You configure which product attributes are searchable/displayed.

Example: Config in `config/packages/pimcore_ecommerce.yaml`

```
pimcore_ecommerce_framework:
  index_services:
    mysql:
      type: MySql
      config:
        tablename: ecommerce_index
  workers:
    default:
      type: default
```

Run:

```
bin/console ecommerce:indexservice:bootstrap
```

👉 This creates the product index table.

🔷 6. Add Products to Index

In Pimcore Admin:

- Go to **Marketing → E-Commerce → Index**.
- Select “Add Products to Index” → choose your Product objects.

✅ Now they’re available for catalog browsing.

🔷 7. Create a Catalog Page

1. Go to **Documents** → **Add Page** → name = `catalog`.
2. In your Twig template (`catalog.html.twig`), fetch products from the index:

```
{% set products = pimcore_ecommerce_index('default').getProducts() %}

<h1>Product Catalog</h1>
<ul>
  {% for product in products %}
    <li>
      <h2>{{ product.name }}</h2>
      <p>SKU: {{ product.sku }}</p>
      <p>Price: {{ product.price }} €</p>
      <img src="{{ product.image | pimcore_image({'thumbnail': 'catalog'}) }}">
    </li>
  {% endfor %}
</ul>
```

🔷 8. Result

- Visit `/catalog` → You’ll see a simple **product listing page** with:
  - Name
  - SKU
  - Price
  - Image

🔷 9. Next Steps (Optional)

- Add **filters/facets** (e.g., by category, price range).
- Enable **cart & checkout** (via CartManager).
- Integrate with **ERP/CRM** for stock & pricing.

✅ You now have a **basic product catalog** powered by Pimcore's eCommerce framework.

🔷 1. Bundles vs Plugins

- **Bundles (Symfony)**
  - Standard Symfony packages.
  - Installed via Composer.
  - Can add services, controllers, templates, commands.
  - Examples: API Platform, Doctrine Extensions.
- **Plugins (Pimcore)**
  - Pimcore-specific way to extend admin UI or core features.
  - Can add custom admin menus, dashboard widgets, object field types, workflows.
  - Stored in `/bundles` or `/var/plugins`.

👉 Rule of thumb:

- **Bundles** = use Symfony ecosystem (business logic).
- **Plugins** = extend Pimcore-specific admin/editor features.

🔷 2. Example: Create a Custom Symfony Bundle in Pimcore

We'll build a **"Product Review" feature** where users can leave reviews for products.

Step 1: Generate a Bundle

Run in project root:

```
bin/console pimcore:bundle:generate
```

- Bundle name: `AppProductReviewBundle`
- Namespace: `App\ProductReviewBundle`
- Location: `/bundles/AppProductReviewBundle`

This generates boilerplate files.

Step 2: Add a Controller

Create a controller inside the bundle: `/bundles/AppProductReviewBundle/src/Controller/ReviewController.php`

```php
<?php

namespace App\ProductReviewBundle\Controller;

use Symfony\Bundle\FrameworkBundle\Controller\AbstractController;
use Symfony\Component\HttpFoundation\Request;
use Symfony\Component\HttpFoundation\Response;
use Symfony\Component\Routing\Annotation\Route;

class ReviewController extends AbstractController
{
    #[Route('/reviews/submit', name: 'submit_review', methods: ['POST'])]
    public function submit(Request $request): Response
    {
        $productId = $request->get('productId');
        $review = $request->get('review');

        // Normally you'd persist to DB — simple log for now
        file_put_contents(
            PIMCORE_PROJECT_ROOT . '/var/reviews.log',
            "Product {$productId}: {$review}\n",
            FILE_APPEND
        );

        return new Response('Review submitted for product ' . $productId);
    }
}
```

Step 3: Register Routes

In the bundle, Pimcore auto-loads routes if you configure them in: `/bundles/AppProductReviewBundle/config/routes.yaml`

```
submit_review:
  path: /reviews/submit
  controller: App\ProductReviewBundle\Controller\ReviewController::submit
```

Step 4: Use It in Frontend

In a Twig template for a Product Detail Page (`product.html.twig`):

```
<h2>Leave a Review</h2>
<form method="POST" action="{{ path('submit_review') }}">
    <input type="hidden" name="productId" value="{{ product.id }}">
    <textarea name="review"></textarea>
    <button type="submit">Submit Review</button>
</form>
```

Step 5: Enable the Bundle

Register your bundle in `config/bundles.php`:

```
App\ProductReviewBundle\AppProductReviewBundle::class => ['all' => true],
```

Clear cache:

```
bin/console cache:clear
```

✅ Now, users can submit reviews, and they'll be logged in `var/reviews.log`. Later, you could persist them to a **Doctrine entity** or **Pimcore Data Object**.

🔷 3. Example: Pimcore Plugin Extension

If you want to add this feature inside Pimcore Admin:

- Create a custom **Admin Controller**.
- Add a **menu item** under “Products → Reviews”.
- Display submitted reviews in a **grid view** for editors.

🔷 4. Benefits

- **Bundles** = Symfony power (custom logic, APIs, integrations).
- **Plugins** = Deep Pimcore integration (UI, workflows, DAM, PIM features).

✅ With this, you’ve seen how to extend Pimcore both as a Symfony developer (Bundles) and as a Pimcore specialist (Plugins).
